

```
class Vehiculo {
```

```
def moverse(log):
```

```
| Vehiculo |  
+-----+  
| + moverse()  
| + detenerse()  
+-----+
```

```
| Coche | |Bicicleta|  
+-----+ +-----+
```

```
| + moverse() | + moverse()  
| + detenerse() | + detenerse()  
+-----+ +-----+
```

Guía: Diagrama de clases para modelar la solución: Implementación en C++



El siguiente documento ha sido elaborado tomando fragmentos de documentos previamente publicados, así como imágenes y ejercicios, sobre los cuales se han realizado traducciones libres y adaptaciones con el propósito de desarrollar el material para el curso de Fundamentos de Programación Orientada a Objetos de la Universidad del Valle. Se ha respetado la autoría original de los materiales, y se han proporcionado las referencias correspondientes dentro del documento. El uso de este material se restringe estrictamente al marco académico y formativo de los estudiantes que cursen la asignatura, y no tiene fines comerciales. Queda prohibida su copia o distribución fuera del contexto de dicho curso.

Diagrama de clases para modelar una solución: Implementación en C++

El diagrama de clases es una herramienta clave en la programación orientada a objetos, útil para organizar y visualizar una forma particular de resolver un problema. Así como también representar los elementos de un sistema y cómo se relacionan entre sí. Imagine que quiere construir un sistema como una aplicación de banco o un juego: el diagrama de clases le ayudaría a desglosar los diferentes componentes del sistema en "clases" y definiendo sus atributos y métodos que cada clase contiene.

Cada clase representa una entidad con sus características. Por ejemplo, un "Cliente" en una aplicación bancaria puede tener atributos como: nombre, número de cuenta, y saldo, con acciones como "depositar" o "retirar". Con este diagrama, los programadores pueden verificar las relaciones entre clases, lo cual permite tener un diseño estructurado y evitar errores durante la codificación.

Las relaciones entre clases permiten modelar cómo interactúan las distintas partes de un sistema orientado a objetos. Existen varios tipos de relaciones como, la herencia, la asociación y la composición, que permiten estructurar el comportamiento y la conexión entre clases de manera efectiva. Por ejemplo, la herencia establece una jerarquía entre clases, permitiendo que una clase hija hereda atributos y métodos de una clase padre. Esto es útil para crear estructuras genéricas, como una clase "Vehículo" de la cual derivan clases más específicas como "Coche" o "Moto".

La asociación, por otro lado, representa una relación entre dos clases sin jerarquía, como la relación entre una clase "Cliente" y una clase "Cuenta", donde ambas interactúan pero



Guía: Diagrama de clases para modelar la solución: Implementación en C++

una no depende de la otra. Estas relaciones, bien definidas en el diagrama de clases, guían la implementación en C++ para construir un sistema cohesionado y estructurado.

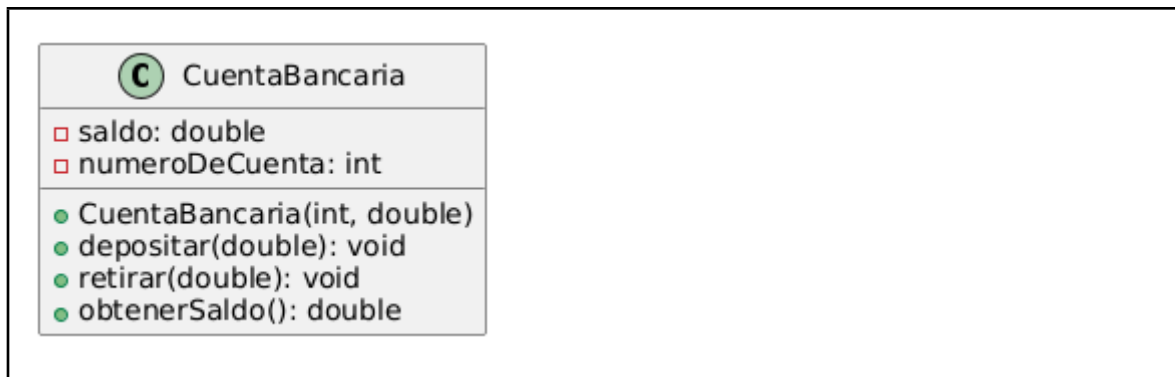
Por otra parte, las relaciones entre objetos ocurren en tiempo de ejecución y representan las conexiones dinámicas entre las instancias de las clases. A diferencia de las relaciones entre clases, que son estructurales (y en tiempo de diseño), las relaciones entre objetos son prácticas y reflejan cómo los objetos específicos interactúan en un contexto particular. Por ejemplo, una aplicación de compras en línea, un objeto "Usuario" puede enviar un mensaje a un objeto "Carrito" para agregar un producto, simplemente llamando al método `agregarProducto()` del carrito. De esta forma, el "Usuario" no necesita conocer los detalles de cómo el "Carrito" maneja la lista de productos; solo sabe que puede enviar el mensaje adecuado para que la acción se realice. En C++, este paso de mensajes se implementa mediante llamadas a métodos de otros objetos, que funcionan como interfaces de comunicación.

En C++, el diagrama de clases se traduce directamente a la implementación mediante el uso de la palabra reservada, así el diagrama es una forma visual de la implementación. En este diagrama, los conceptos del paradigma orientado a objetos se ven reflejados en: abstracción, encapsulamiento, herencia y polimorfismo. También, se modelan las relaciones entre clases, objetos y patrones de diseño tales como clase controladora, entre otros. Al basar la implementación en el diagrama de clases, el programador puede construir el sistema de una manera clara y modular, lo que facilita la comprensión y el mantenimiento del código.

Clase: propiedades y función miembro

En programación orientada a objetos, una clase es una estructura que define tanto los datos como las funciones miembro que operan sobre esos datos. Los datos se representan por atributos, que definen las características de la clase, mientras que las funciones miembro permiten manipular esos atributos y realizar operaciones específicas. Esta combinación facilita el encapsulamiento, es decir, la capacidad de ocultar los detalles internos de la clase y exponer sólo los métodos necesarios para interactuar con ella. Por ejemplo, en una clase `CuentaBancaria`, los atributos como `saldo` o `numeroDeCuenta` representan los datos, mientras que las funciones miembro, como `depositar()` y `retirar()`, permiten modificar el saldo de acuerdo con las acciones de depósito o retiro. Esto hace que la clase sea una entidad completa y autónoma que puede gestionar sus propios datos y comportamientos de manera controlada, ver el siguiente recuadro.

Guía: Diagrama de clases para modelar la solución: Implementación en C++



Recuadro 1. Ejemplo de la clase cuenta bancaria

En el diagrama de clases presentado, se utilizan varias convenciones de notación UML (*Unified Modeling Language*), que son estándares para representar las estructuras y relaciones en sistemas orientados a objetos. A continuación, se explican estas convenciones:

Compartimentos: un diagrama de clases generalmente tiene tres compartimentos, cada compartimento se encuentra separado por una línea horizontal para mantener claridad y estructura.

1. **Nombre de la Clase:** parte superior.
2. **Atributos:** centro.
3. **Métodos o Funciones Miembro:** inferior.

Nombre de la Clase

El nombre de la clase identifica la entidad o el concepto que representa en el sistema, el formato y la ubicación del nombre de la clase, como ``CuentaBancaria``, se coloca en la parte superior del diagrama y se centra. Se escribe con letra en negrita y capitalizada en la primera letra de cada palabra.

Atributos

Los atributos o datos de la clase se listan en el segundo compartimento. En este caso:

- ``saldo: double``
- ``numeroDeCuenta: int``

Visibilidad: La convención de visibilidad (``+`` y ``-``) es esencial para mostrar el encapsulamiento. Este principio de la programación orientada a objetos permite que una clase controle el acceso a sus atributos, exponiendo solo lo necesario mediante métodos públicos (como ``obtenerSaldo()``), mientras mantiene los datos y operaciones como privados.



Guía: Diagrama de clases para modelar la solución: Implementación en C++

- (menos) para privado: significa que el atributo sólo es accesible desde dentro de la clase misma.

+ (más) para público: aunque no lo usamos en los atributos en este ejemplo, indica que el atributo sería accesible desde fuera de la clase.

Tipo de Dato: después del nombre de cada atributo, se especifica su tipo de dato (`'double'`, `'int'`, etc.) con dos puntos **:** para indicar qué tipo de información almacena el atributo.

Métodos o Funciones miembro

Los métodos o funciones miembro se listan en el tercer compartimento. Ejemplos en el diagrama:

```
- `CuentaBancaria(int, double)`  
- `depositar(double): void`  
- `retirar(double): void`  
- `obtenerSaldo(): double`
```

Visibilidad: al igual que con los atributos, los métodos usan símbolos de visibilidad:

+ (más) para público: permite que el método sea accesible desde fuera de la clase.

- (menos) para privado aunque en este ejemplo no hay métodos privados.

Parámetros: dentro de los paréntesis, se especifican los parámetros necesarios para cada método, indicando el tipo de datos de cada parámetro, como en `'depositar(double)'`.

Tipo de Retorno: después de los paréntesis de cada método, el tipo de datos de retorno se indica tras dos puntos **:**. Por ejemplo, `'obtenerSaldo(): double'` devuelve un valor de tipo `'double'`.

Constructor: el constructor de la clase, `'CuentaBancaria(int, double)'`, es un método especial que se utiliza para inicializar nuevos objetos de la clase. Se muestra en el diagrama igual que otros métodos, pero se distingue por tener el mismo nombre que la clase y no tener un tipo de retorno.

Estas convenciones ayudan a que los diagramas de clases sean fácilmente comprensibles para cualquier desarrollador familiarizado con UML, mostrando de forma clara cómo están estructuradas las clases y cómo deberían interactuar.

Otro ejemplo, en el siguiente recuadro se presenta la clase muestra partes de una clase real para programar flash chips de memoria en el sistema embebido (familia Am29). Eso contiene varios miembros de datos privados, protegidos y públicos y funciones de los miembros. Tiene un constructor paramétrico (no constructor predeterminado). Sin embargo, no depende de otras clases.

Guía: Diagrama de clases para modelar la solución: Implementación en C++

```
1 // Interface to onboard flash memory programming
2 class Am29_FlashProgrammer
3 {
4 public:
5     enum EFlashBitMask{ k_DQ7_Mask=0x01,
6 k_DQ6_Mask=0x02 };
7
8     using Systemaddress = uint32_t; // 32-bit
9     address bus
10     using Datalen      = uint32_t; // 32-bit
11     address bus
12     using DataWordType = uint16_t; // 16-bit data
13     bus
14
15 private:
16     // system flash base address
17     const Systemaddress fFlashBaseAddress;
18
19 protected:
20
21     // Checks whether a given bit toggles
22     bool ToggleBit_PassControl( EFlashBitMask mask,
23 SystemAddress offsetAddress = 0);
24
25 public:
26     // Class constructor
27     Am29_FlashProgrammer( Systemaddress
28 flashBaseAddress );
29
30 public:
31
32     // This function ERASES the ENTIRE device
33     bool ChipErase( void );
34
35     // This function prog block of the memory
36     Boot. ProgramBlock( Sfat8Wldress addressIgeJash,
37 DatalWordTyA8 * bufAddr, Datalen dataLen
38 );
39
40 };
41
```

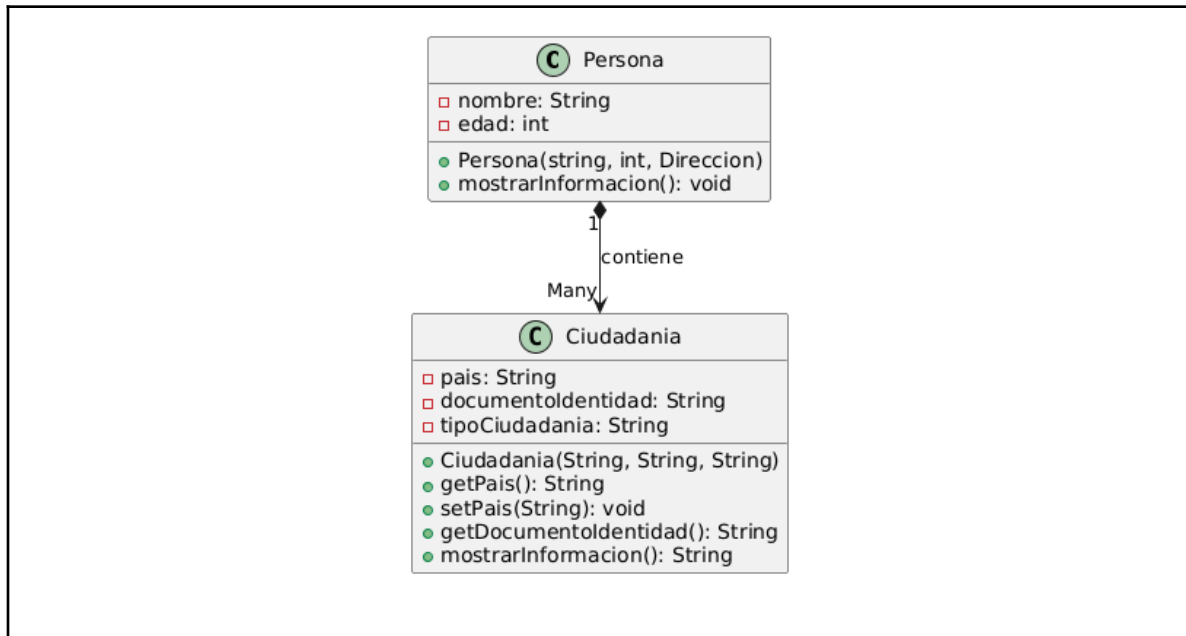
Recuadro 2. Ejemplo Flash

Relaciones entre clases

Hemos visto un ejemplo de clases que contienen datos y funciones miembro. En algunos casos estos datos miembros pueden ser objetos de otras clases. Esta es probablemente la relación más natural entre clases: la composición. La composición es una relación entre clases donde una clase contiene uno o más objetos de otra clase como datos miembros. Esto significa que la clase contenedora ("compuesta") depende de los objetos que posee y, en muchos casos, controla su ciclo de vida. En C++, esta relación se modela al incluir objetos de otras clases como atributos dentro de una clase principal. Imaginemos el ejemplo de una clase **Persona** y una clase **Ciudadanía**. Cada persona tiene una o varias ciudadanías, por lo que la ciudadanía es una parte esencial de la **Persona**. Aquí, **Ciudadanía** es un objeto miembro de **Persona**, representando una relación de

Guía: Diagrama de clases para modelar la solución: Implementación en C++

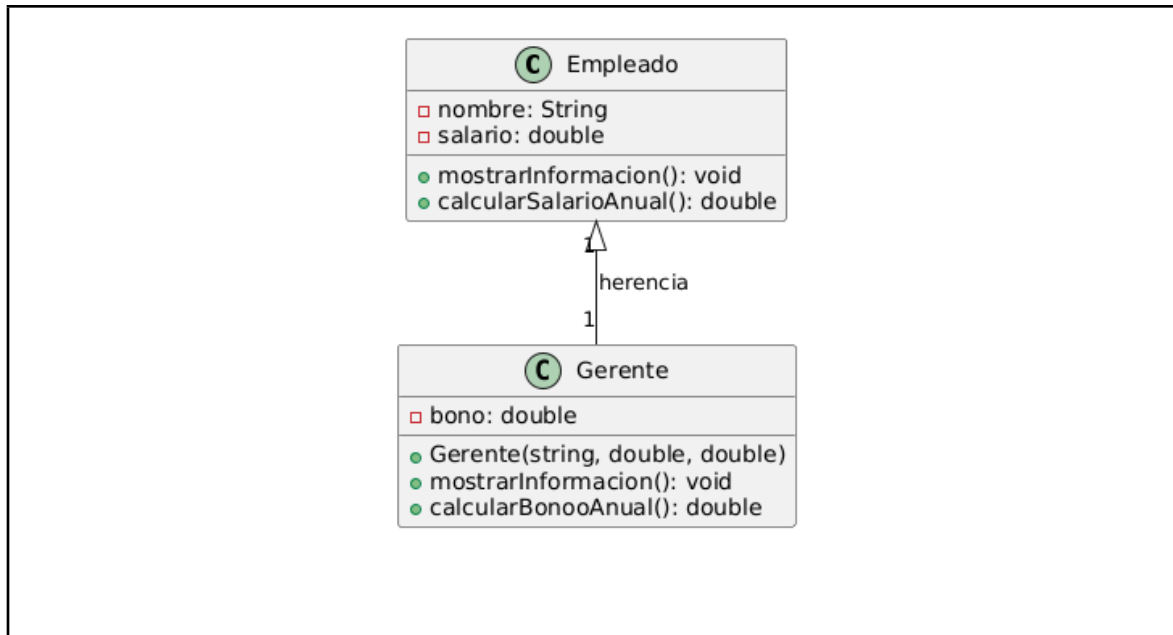
composición porque **Persona** contiene y depende de una instancia de **Ciudadanía**. Si eliminamos un objeto **Persona**, su **Ciudadanía** también deja de existir, lo que refleja el estrecho vínculo entre ambos.



Recuadro 3. Ejemplo de la relación composición

Además de la composición hay otra relación igual de importante: la herencia que permite crear una nueva clase (clase hija o derivada) basada en otra clase existente (clase base o padre). La clase hija hereda los atributos y métodos de la clase base, pero también puede extender (añadir nuevos miembros) o modificar (sobrescribir métodos) el comportamiento de la clase base. Esta relación permite la reutilización de código y facilita la creación de estructuras jerárquicas. Imaginemos una clase base **Empleado** que representa a un empleado genérico en una empresa. Luego, crearemos una clase derivada **Gerente** que hereda de **Empleado** pero agrega y modifica algunas funcionalidades específicas para un gerente.

Guía: Diagrama de clases para modelar la solución: Implementación en C++

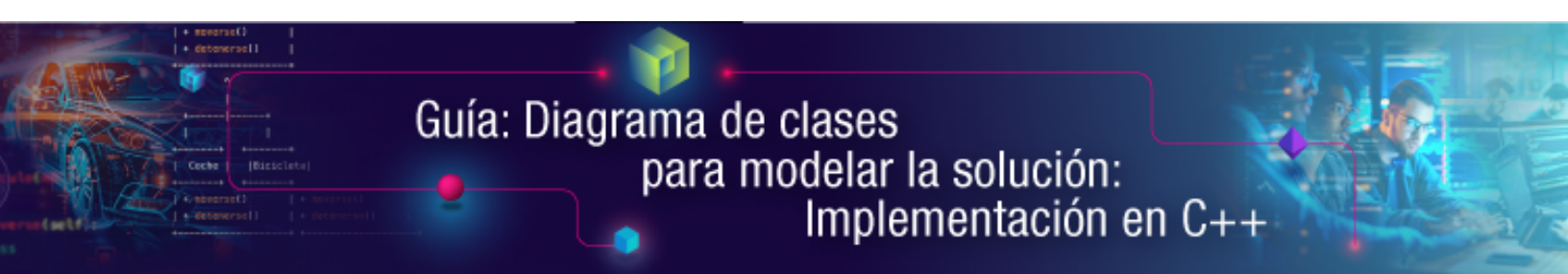


Recuadro 4. Ejemplo de la relación herencia

Explicación del Ejemplo

1. **Clase Base Empleado:**
 - **Atributos:**
 - **nombre:** Nombre del empleado.
 - **salario:** Salario mensual del empleado.
 - **Métodos:**
 - **mostrarInformacion():** Muestra el nombre y salario del empleado.
 - **calcularSalarioAnual():** Calcula el salario anual multiplicando el salario mensual por 12.
 - La palabra clave virtual permite que estos métodos puedan ser sobrescritos en la clase derivada.
2. **Clase Derivada Gerente:**
 - **Atributo Adicional:**
 - **bono:** Un bono anual específico para los gerentes.
 - **Métodos Sobrescritos:**
 - **mostrarInformacion():** Este método se sobrescribe para incluir la información del bono.
 - **calcularSalarioAnual():** Modifica el cálculo del salario anual sumando el bono al salario anual básico.

En este caso, Gerente hereda todos los atributos y métodos de Empleado, pero adapta su funcionamiento para incluir el bono específico de los gerentes. De esta forma, la herencia



Guía: Diagrama de clases para modelar la solución: Implementación en C++

permite reutilizar la estructura de Empleado mientras añade características específicas para Gerente. En el diagrama, la flecha indica que Gerente **hereda** de Empleado, mostrándonos cómo Gerente reutiliza y extiende las funcionalidades de Empleado.

Tener en cuenta que en la relación de herencia hay modificadores de acceso: privado, protegido y público. En el curso sólo utilizaremos el acceso público, un ejemplo de implementación de herencia en C++ se presenta en el siguiente recuadro.

```
1  #include <iostream>
2  using namespace std;
3
4  // Clase Base
5  class Animal {
6  public:
7      // Atributo de la clase base
8      string nombre;
9
10     // Constructor de la clase base
11     Animal(string n) : nombre(n) {}
12
13     // Método de la clase base
14     void hacerSonido() {
15         cout << "El animal hace un sonido." << endl;
16     }
17 };
18
19 // Clase Derivada
20 class Perro : public Animal {
21 public:
22     // Constructor de la clase derivada
23     Perro(string n) : Animal(n) {}
24 }
```

Guía: Diagrama de clases para modelar la solución: Implementación en C++

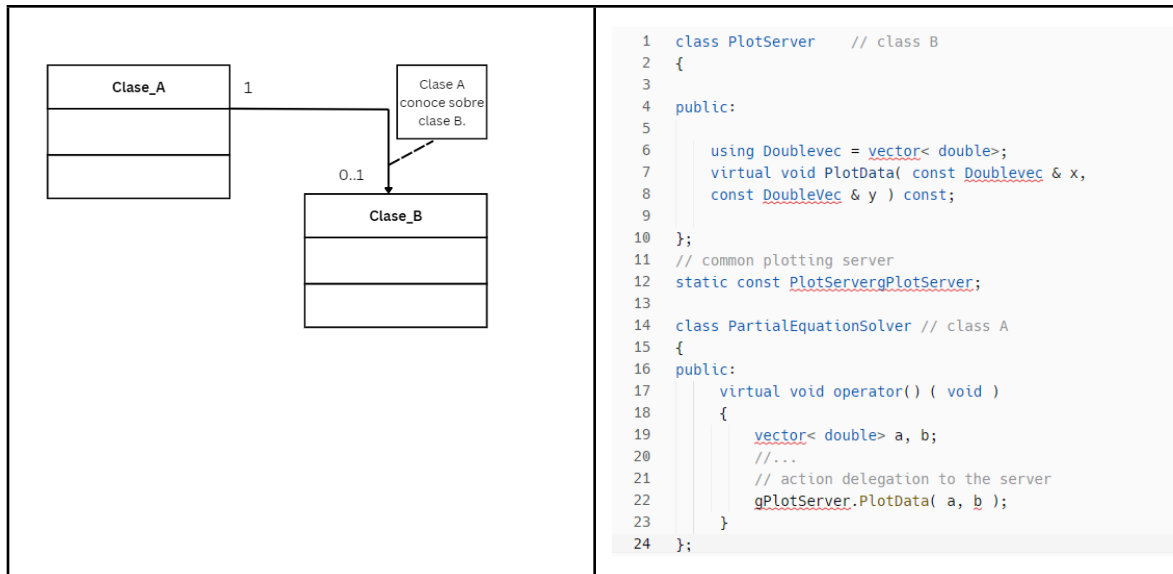
```
25     // Sobrescribir el método de la clase base
26     void hacerSonido() {
27         cout << "El perro " << nombre << " ladra." << endl;
28     }
29 };
30
31 int main() {
32     // Crear un objeto de la clase derivada
33     Perro miPerro("Fido");
34
35     // Llamar al método sobrescrito en la clase derivada
36     miPerro.hacerSonido();
37
38     // También se puede llamar al método de la clase base (si
no está sobrescrito)
39     Animal miAnimal("Genérico");
40     miAnimal.hacerSonido();
41
42     return 0;
43 }
```

Recuadro 5. Implementación de herencia en C++

Relación de clase asociación o relación del tipo conoce

En este tipo de relación la **Class A** "conoce" sobre la Clase B, por lo que la Clase A puede llamar a los miembros de **Class B**. Esta relación se indica con una flecha de la Clase A a la Clase B. En la relación conoce, las relaciones pueden ser, unidireccional o bidireccional (flecha bidireccional). Opcionalmente, los extremos de las conexiones pueden contener cantidades. Los objetos de estas clases no dependen unos de otros, pero cambiar la interfaz de uno puede afectar al otro, ver el siguiente recuadro.

Guía: Diagrama de clases para modelar la solución: Implementación en C++



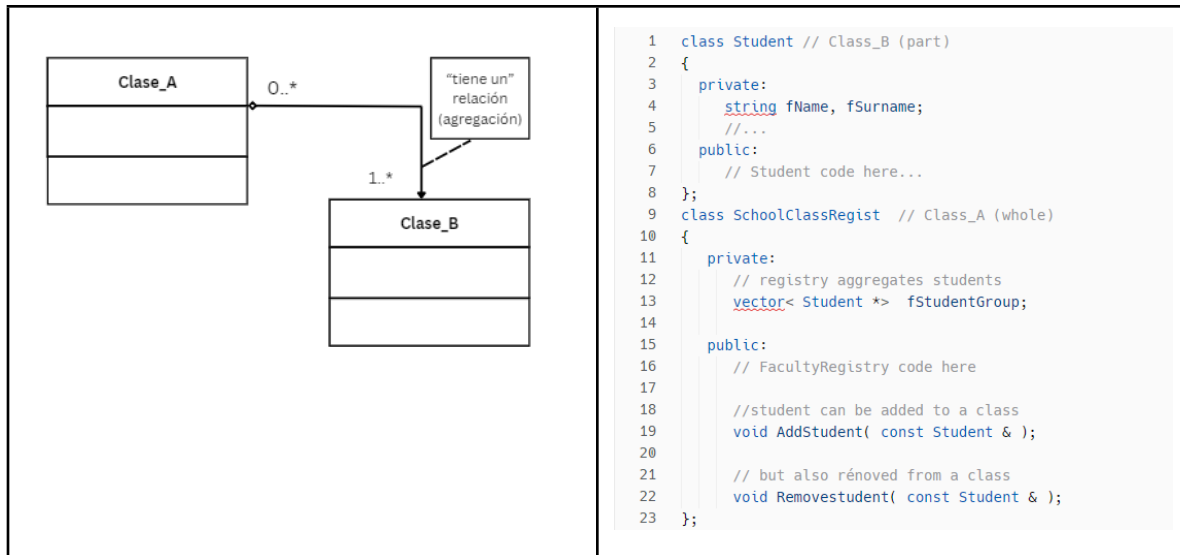
Recuadro 6. Relación del tipo conoce

En el ejemplo las clases tienen una relación flexible y unidireccional. **PlotServer** (**Class_B**) es una clase que facilita los servicios de impresión. **PartialEquationSolver** (**Class_A**) conoce y solicita los servicios de **PlotServer** para imprimir algunos de sus resultados. Pero las dos clases no están vinculadas. Un objeto del **Class_B** debe estar vivo cuando es llamado por un objeto de **Class_A**, lo cual se cumple aquí ya que **gPlotServer** es un objeto estático.

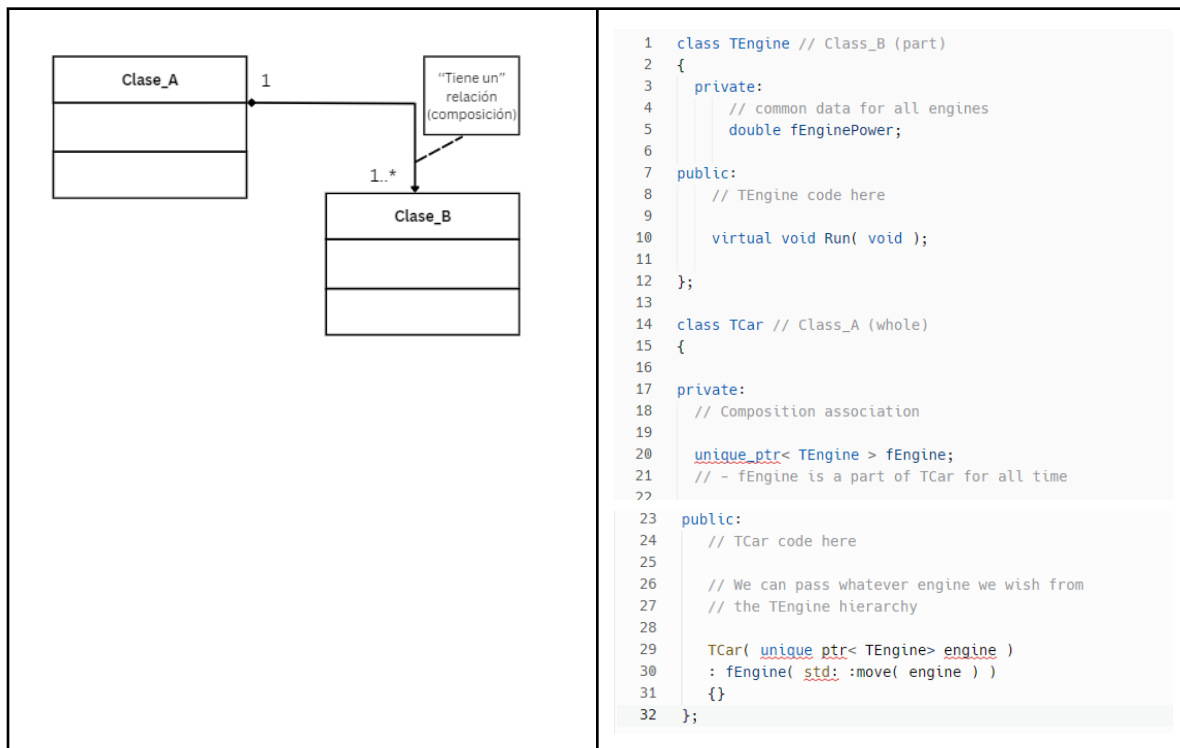
Relación de clase agregación o relación del tipo tiene un

En este tipo de relación un objeto de **Class_A** agrega un objeto de **Class_B**. La relación del tipo agregación también se conoce como "Todo-parte" o "tiene-un". Esta es una versión más laxa de la relación todo-parte en la que no hay dependencia de por vida entre un objeto de **Class_A** (todo) y un objeto (s) de clase B (parte). Por ejemplo, la clase **FacultyRegistry** agrega objetos de la clase **Student**. Sin embargo, ambos pueden existir el uno sin el otro. **Class_B** puede pertenecer a uno o más objetos, por lo tanto, su multiplicidad puede ser 1 o más (denotado como 1..*). Pero un todo, **Class_A**, puede sumar 0 o más par (denotado como 0..*).

Guía: Diagrama de clases para modelar la solución: Implementación en C++



Recuadro 7. Relación del tipo tiene-un agregación



Recuadro 8. Relación del tipo tiene-un composición

Un automóvil debe tener un motor incorporado. Por lo tanto, la clase **TCar** (Class_A) posee un objeto de la clase **TEngine** (Clase_B). Lo importante aquí es que **TCar** y sus objetos **TEngine** están estrictamente relacionados con la vida de la **Class_A**. Es decir, si

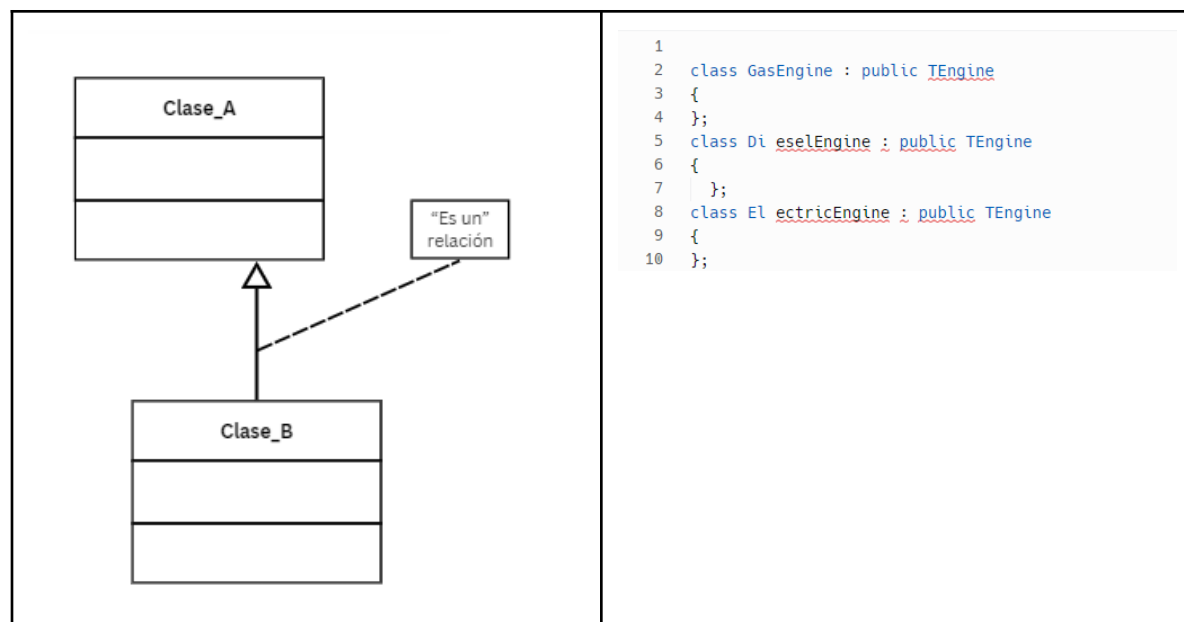


Car se destruye, también se destruye su motor. En la práctica, el enlace de **Class_A** a los objetos de la **Class_B** se realiza mediante una referencia o un puntero, ver recuadro anterior.

Relación de clase herencia o relación del tipo es un o relación del tipo es parecido a un

La relación de herencia o "es-un" permite la especialización de la clase base, en este sentido la **Class_B** se deriva de la clase base **class_A**. Así, **Class_B** es del tipo de **class_A** de ahí su nombre "es-un". La **Class_B** hereda características y funciones miembro de la **Class_A**. Además, puede agregar sus propios miembros y cambiar su comportamiento. También es posible que una clase se derive de más de una clase base (herencia múltiple). La clase base se la conoce como superclase o clase madre o clase principal. La clase derivada es frecuentemente llamada como subclase o clase secundaria.

Para gestionar el encapsulamiento de la clase base, se utiliza el filtro de acceso **privado**, **protected** y **publico** en la clase base. El filtro de acceso limita el acceso de la clase derivada, así si en la clase base una característica o una función miembro está privada, la clase derivada no lo puede acceder, sin embargo, si está en la sección **protected** esta será similar al acceso público para la clase derivada, pero privada para el resto de clases que no heredan de la clase base. En el caso de estar en el acceso público es público para todos.



Recuadro 9. Relación de herencia

La herencia nos permite proporcionar objetos especializados en comparación con la clase base, **Gas** es una clase base (**Class_A**), mientras que **GasEngine**, **DieselEngine** y



Guía: Diagrama de clases para modelar la solución: Implementación en C++

ElectricEngine son todas especializaciones derivadas (**Class_B**). Las clases derivadas heredan todos los miembros y la funcionalidad de la clase base. También, pueden cambiar el comportamiento definido en la base mediante el uso de funciones virtuales. Además, cada clase derivada se puede usar en lugar de la clase base (el principio de sustitución de Liskov). Por lo tanto, **DieselEngine** o **ElectricEngine** se pueden proporcionar al constructor de la clase **TCar**.

Multiplicidad de relaciones

La multiplicidad de relaciones en el contexto de los diagramas de clases de programación orientada a objetos se refiere a la cantidad de instancias de una clase que pueden estar asociadas con una instancia de otra clase. Esta relación se expresa mediante notaciones como ``0..1``, ``1..1``, y ``0..*``, entre otras. Por ejemplo:

- ``0..1`` indica que puede no haber ninguna instancia (0) o una única instancia (1) de la clase asociada.
- ``1..*`` significa que debe haber al menos una instancia (1) y puede haber múltiples instancias (n).
- `0..*`` indica que una clase puede estar asociada con (0) o muchas instancias de otra clase.

Esta especificación es clave para definir claramente cómo interactúan las clases entre sí, ya que determina si una clase puede existir sin su relación con otra y cuántas instancias son necesarias para mantener la integridad del modelo. La multiplicidad ayuda a los desarrolladores a comprender mejor la estructura del sistema y a implementar correctamente la lógica de negocio en el código, en la siguiente tabla se presentan los tipos de multiplicidades y los tipos de flechas a utilizar en cada caso, ver la siguiente tabla.

Multiplicidad de relaciones	Descripción	Miembro de la clase visibilidad	Descripción	Tipo de relación	Descripción
0	Ninguna instancia	+	Público		Asociación
0..1	Ningún caso o exactamente uno	-	Privado		Dependencia
1	Exactamente una instancia	#	Protegido		Herencia
0..*	Cero o más instancias	/	Derivado (herencia)		Agregación
1..*	Una o más instancias	~	Paquete (bibliotecas)		Composición

Tabla 1: Multiplicidad de relaciones diagrama de clase